```
> restart:

  # Compute partition numbers by Euler's pentagonal recurrence.
  BuildPartitionTable := proc(Nmax::nonnegint)
      local Ptab, nn, r, g, sign, total;

      Ptab := Array(0..Nmax, datatype=anything, fill=0);
      Ptab[0] := 1;

      for nn from 1 to Nmax do
          total := 0;
          r := 1;
          sign := 1;

          while true do
              g := iquo(r*(3*r-1), 2);

              if g > nn then
                  break;
              end if;

              total := total + sign*Ptab[nn-g];

              if g+r <= nn then
                  total := total + sign*Ptab[nn-g-r];
              end if;

              r := r+1;
              sign := -sign;
          end do;

          Ptab[nn] := total;
      end do;

      return Ptab;
  end proc:

  # Compute M(u,nn) using (2.4) with iota=1.
  # Negative partition arguments contribute zero.
  # All calls in the verification have nn>=72.
  CrankCoeff := proc(u::nonnegint, nn::posint, Ptab)
      local r, x, term, total, sign;

      total := 0;
      r := 1;
      sign := 1;
```

```
    while true do
        x := nn-r*u-iquo(r*(r-1), 2);

        if x < 0 then
            break;
        end if;

        term := Ptab[x];

        if x >= r then
            term := term-Ptab[x-r];
        end if;

        total := total + sign*term;
        r := r+1;
        sign := -sign;
    end do;

    return total;
end proc:

# Exact number of required determinants with 72 <= n <= H.
Count42Through := proc(H::nonnegint)
    local q, rem3;

    if H < 72 then
        return 0;
    elif H <= 108 then
        return iquo((H-72)*(H-71), 2);
    end if;

    q := iquo(H, 3);
    rem3 := irem(H, 3);

    return iquo(3*q*(q-1), 2)+(rem3+1)*q-1260;
end proc:

# Verify every required determinant in the chosen n-block.
Verify42 := proc(nFirst::posint, nLast::posint)
    local Ptab, nn, m, bound, left, mid, right,
          det, checked, expected;

    if nFirst < 72 or nLast < nFirst or nLast >= 10^6 then
        error "require 72 <= nFirst <= nLast < 10^6";
    end if;
```

```
    expected := Count42Through(nLast)-Count42Through(nFirst-1);

    printf("Computing p(0),...,p(%a).\n", nLast);
    Ptab := BuildPartitionTable(nLast);

    printf("Partition table ready; %a determinants to check.\n",
           expected);

    checked := 0;

    for nn from nFirst to nLast do
        bound := min(nn-72, iquo(nn, 3));

        # Empty m-ranges require no comparison.
        if bound >= 1 then
            left := CrankCoeff(0, nn, Ptab);
            mid := CrankCoeff(1, nn, Ptab);

            for m from 1 to bound do
                right := CrankCoeff(m+1, nn, Ptab);
                det := mid^2-left*right;

                if det < 0 then
                    error "FAIL: n=%1, m=%2, determinant=%3",
                          nn, m, det;
                end if;

                checked := checked+1;

                # Reuse the two neighboring coefficients.
                left := mid;
                mid := right;
            end do;
        end if;

        if irem(nn, 1000) = 0 or nn = nLast then
            printf("Completed n=%a; checked=%a.\n", nn,
checked);
        end if;
    end do;

    if checked <> expected then
        error "incomplete verification: checked=%1,
expected=%2",
              checked, expected;
```

```
      end if;

      printf("PASS: %a <= n <= %a; checked %a determinants.\n",
             nFirst, nLast, checked);

      return [nFirst, nLast, checked, true];
  end proc:

  # Full range in (4.2).
  # Change these two bounds to verify a smaller block.
  nFirst :=93000:
  nLast :=99999:

  result := Verify42(nFirst, nLast):
Computing p(0),...,p(99999).
Partition table ready; 225163167 determinants to check.
Completed n=93000; checked=31000.
Completed n=94000; checked=31197500.
Completed n=95000; checked=62697333.
Completed n=96000; checked=94530500.
Completed n=97000; checked=126697000.
Completed n=98000; checked=159196833.
Completed n=99000; checked=192030000.
Completed n=99999; checked=225163167.
PASS: 93000 <= n <= 99999; checked 225163167 determinants.
>
```